



University of sigmas

Oiowe chady

Adam Biel, Mateusz Jurach, Karol Źeleźnik

2024-03-01

1 Headers

2 Data structures

Headers (1)

.bashrc 11 lines

```
c() {
    g++ -std=c++20 -Wall -Wno-sign-conversion -Wextra -Wshadow -
        Wconversion -Wfloat-equal \
        -fsanitize=address,undefined -DLOCAL -DDEBUG -D_GLIBCXX_DEBUG
        -ggdb3 $1.cpp -o $1
}

nc() {
    g++ -std=c++20 -DLOCAL -O3 -static $1.cpp -o $1
}

alias cp='cp -i'
alias mv='mv -i'
```

.vimrc 7 lines

```
set nocompatible
filetype indent on
set autoindent
set number
syntax on
set ts=4 sw=4
set rnu
```

main.cpp 23 lines

```
#include <bits/stdc++.h>
using namespace std;
#ifdef DEBUG
auto&operator<<(auto &o, pair<auto, auto> p) {o << "(" << p.
    first << ", " << p.second << ")"; return o;}
auto operator<<(auto &o, auto x)->decltype(x.end(), o) {o<<"{";
    for(auto e : x) o<<e<<" "; return o<<""};
#define debug(X) cerr << "["#X"]": " << X << '\n';
#else
#define cerr if(0)cout
#define debug(X) ;
#endif
#define ll long long
#define all(v) (v).begin(), (v).end()
#define ssize(x) int(x.size())
#define fi first
#define se second
#define mp make_pair
#define eb emplace_back

int main() {
    ios_base::sync_with_stdio(false); cin.tie(nullptr);

    return 0;
}
```

gen.cpp

Opis: Dodatek do generowania testów, lub innego losowania. Ziarno można zmieniać np. rng.seed(atoi(argv[1]));

```
mt19937 rng(random_device{}());
int rd(int l, int r) {
    return uniform_int_distribution<int>(l, r)(rng);
}
```

Data structures (2)

wavelet.cpp

Opis: $O(\log^2 \text{ALPH})$ czasowo, $O(n \log \text{ALPH})$ pamięciowo. Constructor przyjmuje ciąg na którym rozpinamy drzewo, rank(l, r, k) zwraca k -tą najmniejszą liczbę na przedziale $[l; r]$. (Uwaga!) pamięć około 262 Mb, dla $N, Q \leq 2 \cdot 10^5$.

```
//funkcja cnt jest w  $O(\log)$  co daje cale operacje w  $O(\log^2)$ ,
//da sie cnt w czasie stalym — do dopisania
struct wavelet_tree {
    const static int ALPH = (1<<18)-1;
    struct Node {
        int low, high, mid;
        vector<int> v;
        Node *left, *right;

        Node() {
            low = 0, high = ALPH;
            left = right = nullptr;
            mid = (low+high)/2;
        }

        Node (const vector<int> &x, const vector<int> &t, int a =
            0, int b = ALPH): low(a), high(b), v(x) {
            mid = ((ll)low+high)/2;
            if(ssize(x) && low != high) {
                vector<int> div[2];
                for(auto e : v) div[t[e]<=mid].eb(e);
                if(ssize(div[1])) left = new Node(div[1], t, low, mid);
                else left = nullptr;
                if(ssize(div[0])) right = new Node(div[0], t, mid+1,
                    high);
                else right = nullptr;
            }
            else {
                left = right = nullptr;
            }
        }
    };

    Node *root;
    wavelet_tree(const vector<int> &b) {
        vector<int> idx;
        REP(i, ssize(b)) idx.eb(i);
        root = new Node(idx, b);
    };

    int cnt(Node *v, int l, int r) {
        if(v == nullptr) return 0;
        return int(upper_bound(all(v->v), r)-lower_bound(all(v->v),
            l));
    }

    int rank(int l, int r, int k, Node *v) {
        if(v->low == v->high) return v->low;
        if(cnt(v->left, l, r) >= k) return rank(l, r, k, v->left);
        else return rank(l, r, k-cnt(v->left, l, r), v->right);
    }

    int rank(int l, int r, int k) {
        return rank(l, r, k, root);
    }

    void clear(Node *v) {
        if(v->left != nullptr) clear(v->left);
        if(v->right != nullptr) clear(v->right);
        delete v;
    }

    ~wavelet_tree() {
```

```
        clear(root);
    }
};

fenwick-tree.cpp
Opis: (Binary Indexed Tree)  $O(\log N)$ , indeksowane od 0. Update w punkcie, zapytanie na przedziale

struct BIT {
    vector<int> tree;
    int N;
    BIT() {N = 0};
    BIT(int n) {
        N = n;
        tree.resize(N);
    }
    void resize(int n) {
        N = n;
        tree.resize(N);
    }
    void update(int p, int x) {
        for (++p; p <= N; p += (p&-p)) tree[p-1] += x;
    }
    int sum(int r) {
        int res = 0;
        for (; r > 0; r -= (r&-r)) res += tree[r-1];
        return res;
    }
    int query(int l, int r) {return sum(r+1)-sum(l);}
    int lower_bound(int sum) {
        int pos = 0;
        for (int pw = 1<<25; pw; pw>=>1) {
            int npos = pos+pw;
            if (npos <= N && tree[npos-1] < sum) {
                pos = npos; sum -= tree[pos-1];
            }
        }
        return pos;
    }
};
```